

# Labyrint-spill

## Beskrivelse av spillet

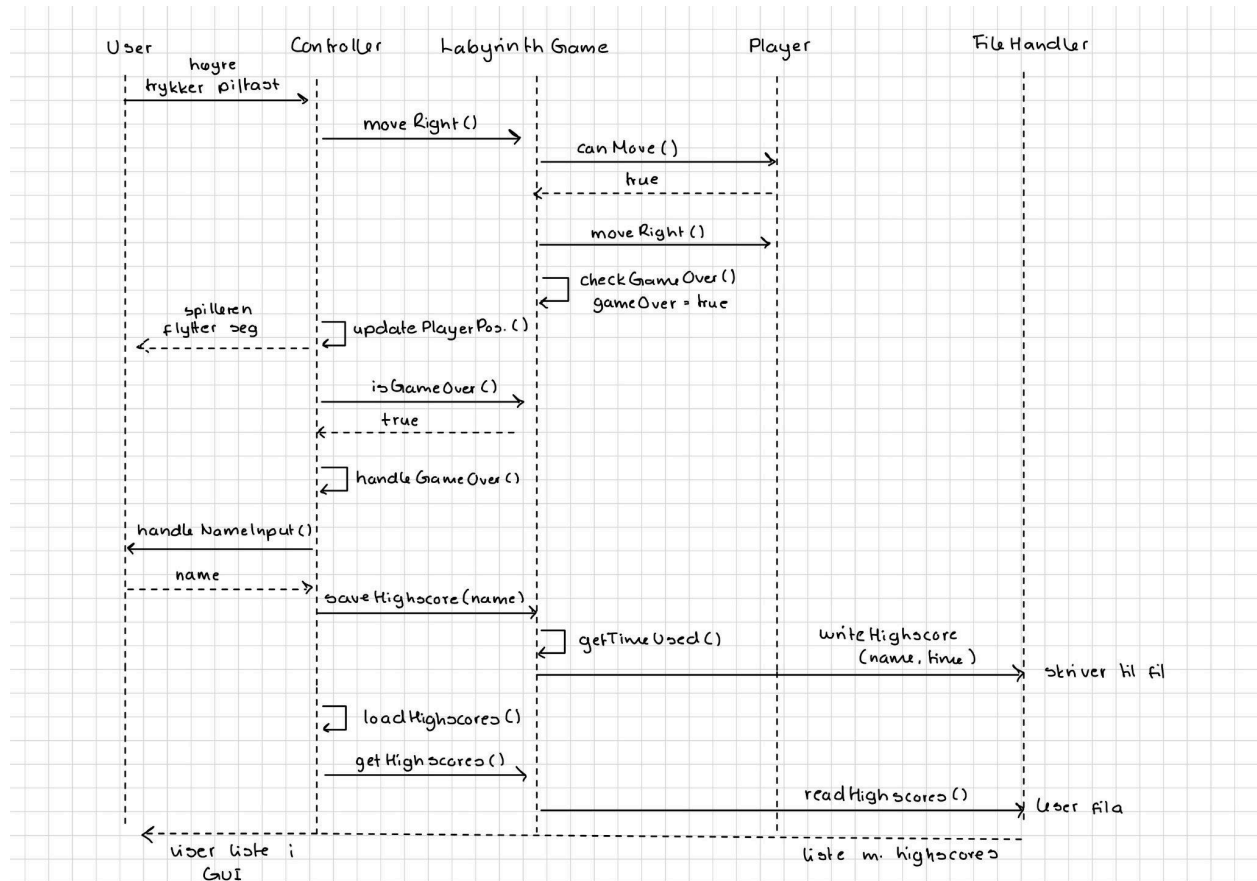
Appen jeg har laget er et labyrint-spill hvor du som spiller navigerer deg gjennom en labyrint fra start til mål ved bruk av piltastene på tastaturet. Labyrinten leses fra en tekstfil hvor # er brukt for å indikere en vegg og fri bane indikeres av et mellomrom. Spilleren kommer i mål når man har kommet ut som indikeres ved F i tekstfilen. Appen tar tiden på hvor lang tid spilleren bruker på å komme seg ut av labyrinten og når spillet er over kan spilleren skrive inn et navn som lagres sammen med tiden brukt i en highscore-liste.

Appen består av 7 klasser. Klassen `Player.java` holder styr på spillerens posisjon og implementerer interfacet `Movable.java`. `LabyrinthGame.java` holder styr på labyrintstrukturen, hvor spilleren starter, sjekker for kollisjon med vegg og om spilleren er i mål. `LabyrinthController.java` er kontrolleren og `LabyrinthApp.java` er view-filen som laster xml-filen. Klassen `LabyrinthFileHandler.java` håndterer fillesing og filskriving av labyrinten og highscore. Det ble også opprettet en egen `Highscore.java`-klasse for å representere en highscore-entry i spillet.

Det var tenkt å også implementere "dødelige prikker" som beveger seg frem og tilbake og som spilleren må unngå (inspirert av Pac-Man), men det blir heller en fremtidig oppdatering av appen.

## Diagram

Sekvensdiagrammet under viser hva som skjer når brukeren beveger spilleren og når spillet avsluttes. Det antas at labyrintfilen er gyldig og har korrekt format og at spilleren starter på en gyldig posisjon (altså ikke på en vegg).



## Spørsmål

Hvilke deler av pensum som dekkes

Prosjektet dekker innkapsling da alle klassene skjuler tilstanden sin ved bruk av private variabler og endres kun gjennom metoder. Grensesnitt dekkes da Player.java implementerer interfacet Movable som definerer metodene moveUp(), moveDown(), osv. Ved å bruke MVC-struktur dekkes delegering siden kontrolleren LabyrinthController delegerer logikk til LabyrinthGame, som igjen delegerer filhåndtering til LabyrinthFileHandler. Unntakshåndtering benyttes flere steder i appen ved at metoder som kan feile (for eksempel ved filoperasjoner) kaster unntaket videre i stedet for å håndtere det selv. Unntakene fanges deretter opp av kontrolleren som håndterer dem ved hjelp av try-catch. På den måten blir koden ryddigere ved at man slipper å håndtere unntaket hvert sted metoden kalles. Filbehandling dekkes av LabyrinthFileHandler som håndterer både skriving til og lesing fra higscore-fila. Comparable dekkes ved at Highscore-klassen implementerer Comparable og metoden compareTo() brukes til å sortere highscore basert på tiden som brukes på å fullføre labyrinthen. Enhetstesting er også dekket gjennom bruk av JUnit-tester som forklart i siste avsnitt under.

## Hvilke deler av pensum som ikke er dekket og hvordan de kunne vært brukt

Arv dekkes ikke, men kunne vært brukt ved å lage en superklasse `GameObject` som `Player`, og også de "dødelige prikkene" når disse senere implementeres, arver fra.

Observatør-observert-teknikken er heller ikke dekket, men kunne muligens blitt brukt i stedet for at kontrolleren aktivt sjekker `isGameOver()`.

## Hvordan koden forholder seg til MVC

Koden følger i stor grad MVC-prinsippet. Modellen (`LabyrinthGame`, `Player`, `Highscore` og `LabyrinthFileHandler`) håndterer logikk og data, mens kontrolleren, `LabyrinthController` håndterer input og oppdaterer GUI-elementer og FXML (view). View viser labyrinten og highscores. I appen er logikken hovedsakelig plassert i modellen, kontrolleren fungerer som et bindeledd mellom GUI og modellen. Kontrolleren inneholder fremdeles noe logikk som f.eks. game over-håndtering, som var vrient å separere helt. Dette er dog et mulig forbedringspotensial.

## Hvordan appen testes

Testingen gjøres med JUnit-tester. Jeg valgte å teste det jeg mener er de viktigste delene av appen hvor potensielle feil vil ha størst påvirkning. Jeg laget fire testfiler som tester logikk i de fire klassene som ikke er kontroller, app-klasse eller interfacet. I fila `FileTest` testes både skrivning til og lesing fra `highscore`-filen. Her kunne det nok også vært relevant å også teste lesing av `labyrinth`-filen. I `HighscoreTest` testes det at `highscore` sorteres korrekt i stigende rekkefølge basert på tid. I `PlayerTest` testes bevegelse med metodene `moveUp()` og `moveRight()` for å sikre at spillerens posisjon faktisk oppdateres når disse metodene kalles. I disse testene er det lagt inn posisjoner som jeg vet ikke er vegg i labyrinten som brukes slik at spilleren skal kunne bevege seg dit. Dette burde kanskje vært løst annerledes og mer generelt for å kunne teste på andre labyrinter, men siden dette prosjektet kun har én labyrint fungerer testen til sitt formål. Den siste testfila `LabyrinthGameTest` tester at spilleren ikke kan forflytte seg på utsiden av labyrinten, at spilleren ikke kan forflytte seg gjennom en vegg, ikke kan starte spillet i en vegg-posisjon og at spilleren faktisk beveger seg. Jeg testet kun bevegelse til høyre og de andre retningene burde nok også vært testet for å dekke flere mulige fallgruver.

## KI-deklarasjon

Selv om det er fristende å spørre KI med en gang man sitter fast har jeg prøvd å google meg frem og sjekke Stack Overflow når jeg støter på problemer for å lære mest mulig. Jeg kom langt med det, men av og til brukte jeg ChatGPT til debugging. Jeg brukte ChatGPT for å finne ut hvordan jeg kunne unngå at når jeg beveger spilleren oppover en gang vil neste gang jeg trykker opp-tasten highlighte start-knappen og ikke flytte spilleren lengre opp. Det viste seg at jeg måtte legge til `focusTraversable="false"` på startknappen. Jeg brukte også av og til ChatGPT til å forklare meg konsepter jeg ikke helt hadde forstått enda, slik som unntakshåndtering.

Hadde ikke helt skjønt hvorfor det var slik at noen metoder kaster unntak uten å håndtere dem, mens andre håndterer dem umiddelbart med try-catch. Jeg brukte også ChatGTP til å bedre forstå filhåndtering og hvilke metoder som passer til hvilket formål da jeg syntes det ble presentert så mange ulike metoder i forelesning. Dette var for å spare tid og kjappere få en oversikt over de ulike metodene. For mer detaljert om bruken av Files.readAllLines og FileWriter brukte jeg andre kilder. Da jeg bruker Google som nettleser tok jeg av og til inspirasjon og testet løsninger fra AI Overview som dukker opp øverst når man søker, men sjekket kildene på siden som sammendraget var basert på.